4 ABSTRACTION AND MODELLING

OBJECTIVES

- Introduce the concept of abstraction and explain its importance.
- Show the importance of layering abstractions.
- Discuss the dangers and limitations of abstraction.
- Introduce modelling as part of the problem-solving process.
- Show typical types of models available.
- Explain the difference between static and dynamic modelling.
- Show typical uses of models along with examples.

ABSTRACTION

This section introduces the concept of abstraction, a key concept in CT for devising powerful solutions.

From generalisation to abstraction

The previous chapter showed the decompositional approach to problem-solving, whereby a large, complex problem is divided into smaller, simpler sub-problems. The motivation behind it is that solving a series of simple problems is easier than trying to tackle a big complex one.

Following on from that was generalisation, an activity that identifies patterns among those individual sub-problems and simplifies them. The effect is to make the overall solution more manageable and widely applicable. In an example (drawing a simple face), we went from drawing very basic shapes (lines, circles, etc.), to drawing an eye with specific dimensions, and then ended up able to draw an eye of *any* size.

In a sense, generalisation is about hiding details.[21] Construction of the drawing solution started with a handful of low-level 'atomic' details and ended up at a more conveniently abstract level. In our example, we went from speaking in terms of constituent elements (two concentric circles, the inner of which was filled solid), to simply referring to an eye. The resulting generic eye is an example of an **abstraction**.

Abstraction: a way of expressing an idea in a specific context while at the same time suppressing details irrelevant in that context.

The importance of abstractions

> The essence of abstractions is preserving information that is relevant in a given context, and forgetting information that is irrelevant in that context.
>
> (Guttag, 2013)

Abstraction is a key feature of both computer science and computational thinking. Some have gone so far as to describe computer science as 'the automation of abstraction' (Wing, 2014).

The reasoning behind this goes right to the core of what programmers and computer scientists are trying to do; that is, solve real-world problems using computers. They can't magically transport the real world into the computer; instead, they have to describe the real world to the computer. But the real world is messy, filled with lots of noise and endless details. We can't describe the world in its entirety. There's too much information and sometimes we don't even fully understand the way it works. Instead, we create models of the real world and then reason about the problem via these models. Once we achieve a sufficient level of understanding, we teach the computer how to use these models (i.e. program it).

Perhaps if we were hyper-intelligent superbeings with limitless mental capacity, we wouldn't need abstractions, and we could take into account every minute detail of our solutions. I hate to be the one to break it to you, but we're not superbeings; we're an intelligent species of ape with the ability to keep about seven or so pieces of information in working memory at any one time.[22]

Consequently, we're stuck struggling to understand the world via our own models of it.

Examples of abstractions

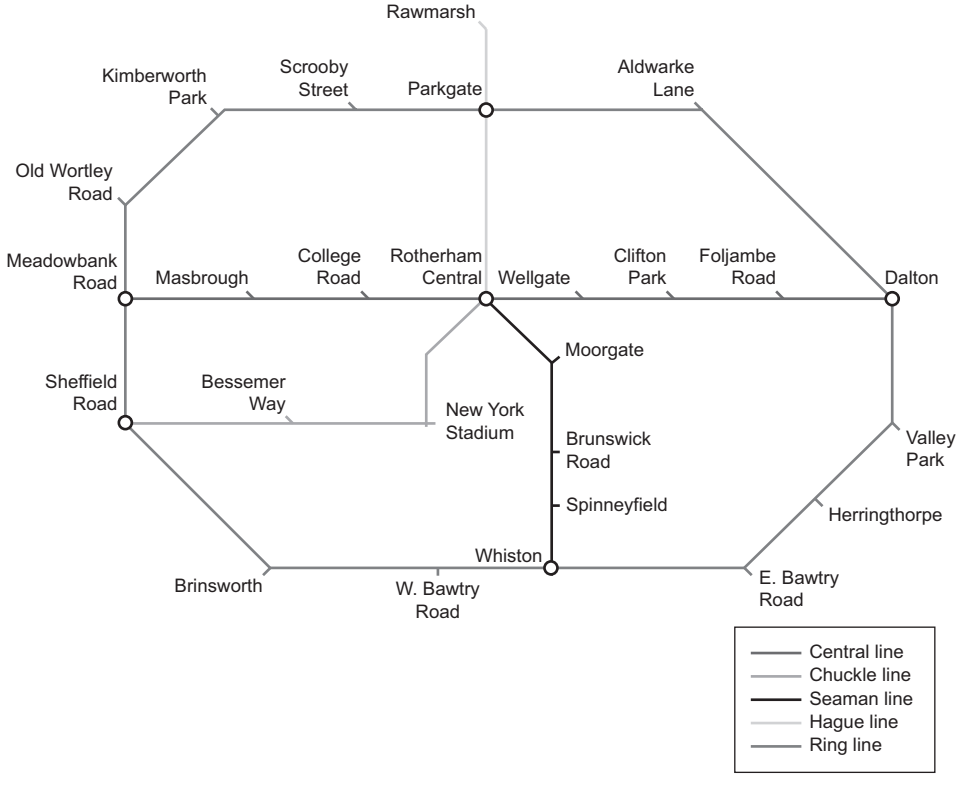
In the previous chapter's drawing example, we created a shape as an abstraction. By itself, the idea of a shape tells you some things – such as that we're dealing with a form that has an external boundary or surface – but other things, like the number of its sides and its internal angles, are unknown details.

A shape might seem an elementary and academic example. In fact, we're surrounded by abstractions in everyday life. Many familiar things are abstract representations of a more detailed reality. One example is an underground metro map. Figure 4.1 shows the underground metro system of Rotherham (a hypothetic system, sadly, as the great and historic town of Rotherham has yet to achieve an underground).

Look at this figure and compare it with the more realistic representation in Figure 4.2. You'll notice it doesn't present a completely accurate picture of reality. The tunnels are not all dead straight at 45-degree angles to one other. The stops along the lines are not

evenly spaced out. However, you wouldn't know that if the only information you had about the underground came from the map.

Figure 4.1 A metro map of the Rotherham underground



Are the map-makers trying to fool us? No, they're leaving out details you don't need to know. It doesn't matter that they left out the U-shaped bend in the line east of Rotherham Central. It doesn't matter that the distances between stations – consecutive and equidistant on the simplified map – are in fact separated by varying distances. It all makes no difference to how you use the underground. What's important is the ordering of the stations, which lines they're on and where those lines converge as interchanges.

Therefore, a typical metro map is an abstraction. It represents certain details (order of stops on the line, intersections of different lines) and suppresses others. As a result, the map is simpler for commuters to use.

If you ask a computer scientist what's so great about abstractions, they may well give you a more technical example, say, an email. Most of the world knows what an email is: a message written on a computer and transmitted via a network to another user. However, this is an abstraction in the same way that 'letter' is an abstraction for a piece of paper with intelligible ink markings on it. There's a lot more underlying detail to an email. If you imagine an email, you probably think of the text in your mail client.

Figure 4.2 A more realistic map of the Rotherham underground[23]

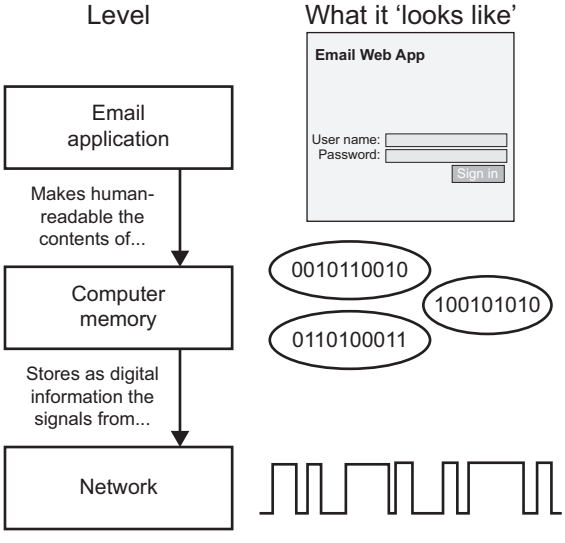
But an email exists as digital information, organised as packets of ones and zeroes in your computer's memory. What's more, when in transit, it exists as a series of electrical impulses travelling through a cable, or as electromagnetic waves hurtling through the atmosphere. If you operated at this level of abstraction, you'd be saying to your friend, 'I'll send you an encoded series of ones and zeroes via electrical impulses over the Internet for you decode into the original human-readable message.'

To which your confused friend might unwittingly reply, 'Why not just send me an email instead?'

Context and layers

That last example of an email hinted towards an important fact about abstractions: context is key. An abstraction of something operates at a certain level of detail and puts a layer over the top to obscure some of the information. You can take any abstraction, peel back the layer and go further 'down' to reveal more information (thus considering it in a more detailed context), or go further 'up', adding more layers to suppress further details about it (a less detailed context). Figure 4.3 can be read as saying 'An email application makes readable the contents of computer memory', as well as 'computer memory stores as digital information the signals from the network'.

Figure 4.3 Layers of abstraction involved in email

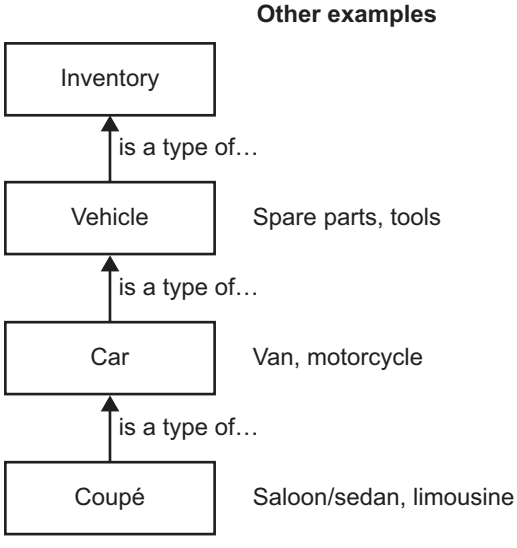


Let's look at a couple of real-world examples to illustrate this. Imagine a vehicle rental company builds a system to manage their inventory. Using the techniques from Chapter 3 (for example, identifying entities, the processes they carry out and the properties they possess), the system designers break down the task and pick out the various concepts. The company has several types of vehicles (cars, vans, motorcycles, etc.) and numerous operations are carried out daily (booking, checking in, checking out, cleaning, road testing, etc.). Once the company identifies all these concepts, it's up to them how to define their abstractions. For example:

- If the company offers a no-frills 'any-old-car-will-do' service, then all the individual cars could be grouped into one abstraction (called, imaginatively, a car).
- Alternatively, if they want to give customers more choice, they could group the cars according to transmission, giving them 'manual cars' and 'automatic cars'.
- All the vans could be classified according to capacity, yielding the abstractions 'small van' or 'large van'.
- When it comes to obtaining a complete listing of everything available at a specific site, no discrimination among all the various types is necessary. In this case, everything – cars, vans, motorcycles – can be dealt with using the single abstraction 'vehicle'.

You can see that each level of abstraction is built up recursively from the levels below.[24] All the Fords and Audis and Nissans, and so on, join to become cars. All the cars join the vans and motorcycles to become vehicles. All the vehicles, spare parts, and tools can be treated collectively as inventory items (see Figure 4.4).

Figure 4.4 Some layers of abstraction in the car rental example



Looking at it this way, the vehicle rental company takes numerous concrete ideas and unifies them into more abstract concepts. However, building abstractions can sometimes go the other way: that is, start with a fairly abstract concept and add more detail as you go along.

Let's take a different example to illustrate this. An online video service like YouTube or Netflix can recommend videos to a user because it has built an abstract model of them. An ideal model of the user would include a perfect replica of their neurons and brain chemistry, so it would know with uncanny accuracy which videos they would like to watch at any moment. Or, failing that, maybe a complete and thorough biography including a run-down of every book, film and TV show they've ever consumed.

However, online video services operate with far less information than that. They know almost nothing about the viewer aside from their video-watching habits (O'Reilly, 2016). This means the video service has to reduce the customer to an abstract video-watching entity. The service knows what kinds of videos the viewer watched in the past and so might push a list of other unseen videos based on that information. It assumes the viewer is most interested in other videos of the same genre.

At the time of writing, online video platforms are in a formative period and such models are constantly changing and rapidly becoming more sophisticated. But, at the moment, however powerful their approaches are, they're still operating by building an abstract model of you, the viewer, based on a tiny subset of information.

Producing better recommendations might mean introducing more detail into their models and taking into account more about the viewer. That would mean having to operate at a lower level of abstraction, which opens up considerations of what is important or

meaningful at that level. A service may decide to divide users by geographical region, so that there isn't just a single concept of a viewer; rather, there is an American viewer, a British viewer, a German viewer and so on. Now, the procedure for building a list of recommendations can be extended with additional steps matching a viewer's region to certain types of films. A German viewer's recommendations would widen to also include some German-language films. But the German-language films wouldn't be included in recommendations outside German-speaking regions.

Exercising caution

Abstractions undoubtedly help, but they should also come with a warning label. Perhaps 'Caution: Abstractions may distract from reality.'

The reason you should be cautious is that abstractions are a way of avoiding details, either by suppressing or deferring them. As in life, when you do that carefully and with good foresight, it can be useful and productive. Doing it recklessly may very well come back to bite you later because it can be easy to get lost in the abstractions. A fuzzy marketing message might imply that a product solves more problems than it really does; a philosophical argument that tries to account for everything might be so lacking in detail that it explains nothing.

In the end, the real question is: how does it work? You can't execute an abstraction. The acid test for a solution depends on how well the concrete implementation works.

Putting abstractions to use

Abstractions are fine for doing things like organising your solution, managing its complexity and reasoning about its behaviour in general terms. But when you actually put the solution into use, it has to work with some level of detail. Put another way: you can't tell a computer to draw a shape, but you can tell it to draw a circle.

To put an abstraction to use requires it to be made concrete somehow. In other words, it is **instantiated** and this requires attention to details. For example, the car rental company may need to list all vehicles according to how soon they need servicing. A generalised procedure for doing this needs some concrete detail to work with. Specifically, the date of its next service must be able to be specified for each vehicle. 'Next service date' is therefore a property of every vehicle. Also, another procedure may involve showing all vans above a specified payload.[25] This procedure only makes sense for vans, so only they need to provide this information when instantiated (see Figure 4.5).

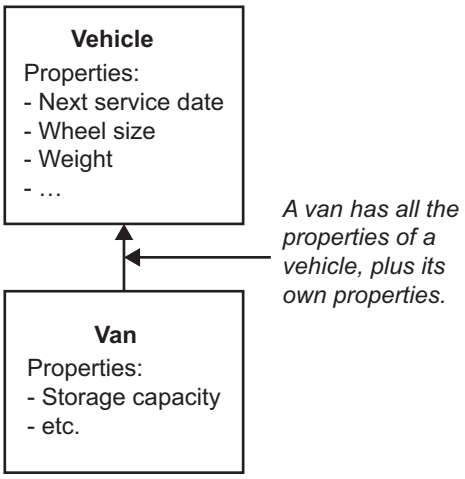
Leaking details

Even after sorting out the underlying details, a risk persists with the use of abstractions: you may have unwittingly ignored a detail that affects the way the abstraction behaves.

Sticking with our automotive theme, the way you operate a car can be considered as an abstraction. A car is a very complicated piece of machinery, but operating it is a simple matter because all those details are hidden behind a few relatively simple control mechanisms: steering wheel, pedals and gear lever. But occasionally, details hidden behind this clean interface affect how the driver uses the car. Revving the accelerator

pedal too high can cause wear on the engine. Braking distance will increase over time as the brake pads become worn. The engine may not start if the weather is too cold.

Figure 4.5 Layers of abstraction including Vehicle and Van



When things like this happen, hidden details have become important to how something works; those details have 'leaked' out of the abstraction. In actuality, no complex abstraction is perfect and all of them will likely leak at some point. But there's no reason to treat this as bad news. The same has been happening in science for centuries.

You may have learned at school (although it might have been phrased differently) that all of science is built on abstractions. Scientists cannot take every single aspect of something into consideration when studying it, so they build simplified models of it: models that ignore friction and air resistance, or models that assume no losses to heat transfer. Technically, such models don't fully reflect reality, but that's not the most important thing. What's important is that they are shown to work.

I know of no general advice to anticipate leaky abstractions ahead of time. However, when yours are shown to leak, you should do as the scientist does: amend your model to take the problem into account. Details that turn out to be important need to be brought explicitly into the model.

MODELLING

'Model'. That word that cropped up quite a lot during the discussion of abstraction. As representations of real-world things that ignore certain details, models are a type of abstraction and an important one in CS. Furthermore, CT has recognised the value of modelling by placing it as one of its fundamental practices. This section explains what models are in the context of computing and how you can put them to use.

Motivation

Abstraction can be a challenging concept to get to grips with. The word itself might be off-putting due to our preconception that abstract things are difficult to understand. Examining specific forms of abstraction can help in understanding it.

In CS, abstraction can take many forms. At its simplest level, abstraction can just mean naming things. For example, a calendar is an abstraction of time; it gives names (day, week, year) to our observations about how long the Earth takes to complete certain movements. Also, the loops and subroutines that we saw in Chapter 3 can be considered abstractions.

By showing a simplified view of things in your problem, models help you to improve your understanding because they allow you to focus only on the relevant parts. This section of the book will explain the fundamentals of modelling in the context of CT. It will show how to use our in-built ability to picture things when developing computer-based solutions.

Basics

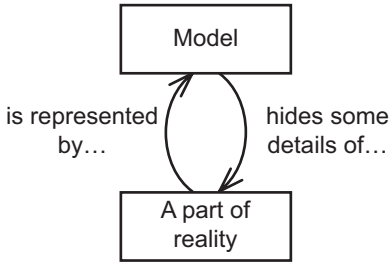
There's nothing particularly magical about models. Put simply, a model shows the entities in your solution and the relationships between them.

We all intuitively understand how to model things because we have imaginations. We make models whenever we sit back and try to picture how a complex thing works, or imagine how a thing will behave in the future. People had been modelling things long before computer science showed up. Some models of the solar system date back to the Ancient Greek world.[26] Economists have modelled the workings of societies throughout the modern era. As already mentioned, scientists build models of reality, from the tiniest atoms to galactic clusters.

All models hide some details of the reality they represent. The things judged irrelevant in a model's context are left out of it (see Figure 4.6). The things remaining play one of two roles in the representation:

- **entities**: the core concepts of the system;
- **relationships**: a connection that relates entities in the system.

Figure 4.6 Abstraction as modelling



Many types of models exist. The same entities in your solution can be modelled over and over again in different ways by using different types of models. Each one offers a different view of the same system, so it's up to you choose the appropriate model(s) depending on which aspects you want to show.

The variety of relationship types is endless. It all depends on what you want the model to describe about your entities. For example, relationships could say things like:

- A belongs to B;
- C contains D;
- E happens before F;
- G occupies the same space as H.

Sometimes, those relationships are directional. Hence:

- B owns A;
- D is contained by C;
- F happens after E.

Entities and relationships are often labelled with additional information if it's necessary for the model. These include:

- **Properties:** discrete pieces of information about that particular entity or relationship, like a station's name on an underground map.
- **Type:** a classification that the entity belongs to. This can tell you lots of things about that entity implicitly. For example, a line between two underground stations relates them by saying they're on the same line; the colour denotes the line's type (e.g. Bakerloo, Circle, Victoria, etc.).
- **Rules:** statements about an entity that must hold true. An underground station might be labelled with a rule saying 'Closed between 10 and 15 November.'
- **Behaviours:** descriptions of actions which may be expressed in natural language or algorithmic form. For example, one section of a metro line might state that trains travel at a slower pace along it.

Static vs dynamic models

Static models give us a snapshot view of the system. They depict entities and relationships at a single point in time. An underground map is a static model. It's a snapshot of the current state of all the stations and lines. True, its state may change in the future (a station may close, another line may be added), but the layout changes so seldom that a static map serves the public just fine.

However, it may be sometimes important to take into account that the world you're modelling changes over time. **Dynamic** models do this. The goal of a dynamic

model is to explain the changes in state as time proceeds. They always involve the following:

- states: descriptions of the entities at specific points in time;
- transitions: a change in state.

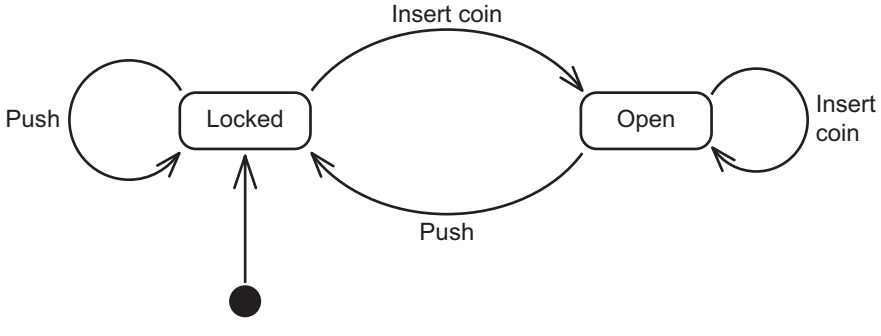
Although there are different varieties of dynamic models, they generally include some or all of the following as well:

- events: things that happen to trigger transitions;
- actions: computations that are carried out as part of a transition.

An example of a simple dynamic system is a turnstile – something that transitions through a series of states as people use it. It can be modelled using a state machine diagram, which depicts the progression of a system through various states (see Figure 4.7).

- At the start point (the solid black circle), the turnstile is in a locked state.
- Inserting a coin is an event. It triggers the transition of a locked turnstile into an unlocked one.[27]
- Pushing an unlocked turnstile (another event) transitions it back to the locked state.
- Pushing a locked turnstile has no effect.

Figure 4.7 State machine diagram of a turnstile



Uses of models

This section shows examples of models assisting in various capacities.

Simplifying and explaining

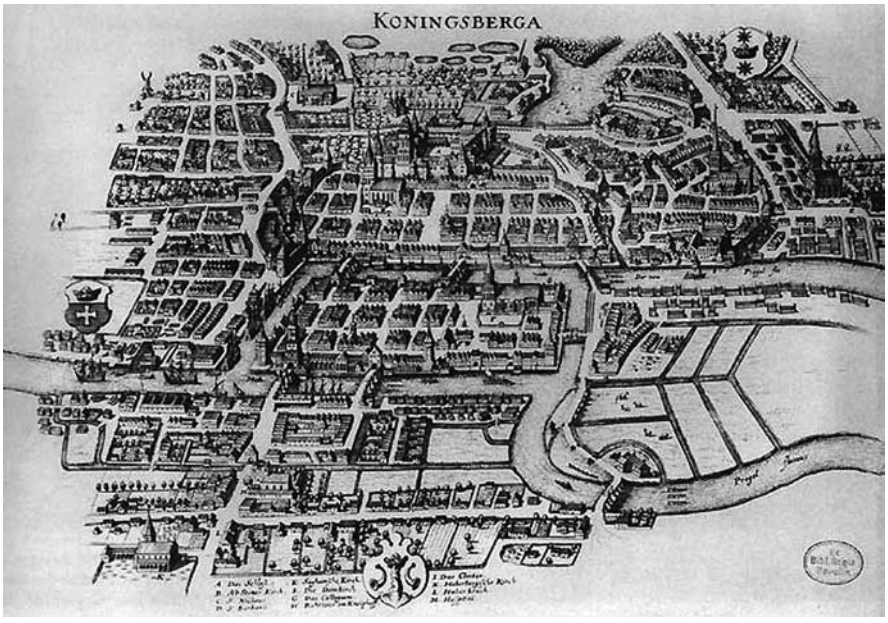
When faced with a problem in a complex world, modelling a part of that world can greatly clarify both the problem and the way towards a solution. A classic example of this dates back a couple of centuries.

Leonard Euler, a Prussian mathematician living in eighteenth-century Königsberg (modern Kaliningrad), was given a challenge by the city's mayor. Seven bridges in the city

crossed the river running through it. The mayor asked Euler whether it was possible to walk across all seven bridges without re-crossing a single one.

To understand the problem, Euler could have studied a map of the city. But the map (see Figure 4.8) was complicated, full of distracting detail. Instead, he reduced it down until only the essential elements remained: the land masses and the bridges. These alone formed the essence of the problem.

Figure 4.8 Map of Königsberg[28]

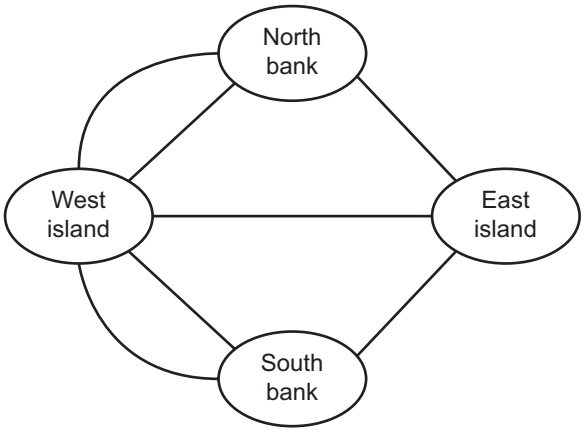


This simplified form, depicted in Figure 4.9, is a type of model called a **graph**. A graph is made up of nodes (which represent entities) and edges (connections between entities that represent relationships). In Euler's model, the nodes represent land masses and the edges represent bridges between land masses.

With this new representation, Euler could focus all his effort on solving the problem. In the process of solving this single concrete question, he came up with some of the fundamental rules of graph theory that can now be applied to any problem when it's represented in graph form.

Just for your interest, the problem has no solution. Euler showed that it's impossible to cross all bridges without re-crossing at least one. His proof was expressed, not in terms of Königsberg, but purely in terms of a graph. So long as every node in the graph has an even number of edges connected to it (with the exception of the start and end node), then the walk is possible. That's because you have to arrive at the node the same

Figure 4.9 Map of Königsberg



number of times you leave it. In this form, Euler's solution applies not just to Königsberg, but to every city in the world.

A graph is just the generic name for this type of model. Quite a few varieties of graph exist. The one in Figure 4.9 is a simple undirected graph. The state machine in the section 'Static vs dynamic models' (p. 66) is also a graph, but in this case it's a directed graph because each relationship flows in one direction.

Executing a representation

Certain dynamic models, once defined, can actually be executed. If that model is a good enough approximation of reality, then executing a model tells you how reality should pan out in the future.

Taking another example from a long-past era, the orrery is a physical model of the solar system (see Figure 4.10). A large ball representing the Sun sits at the centre, while smaller balls (the planets) are perched on arms of varying length. It is, furthermore, a dynamic model. A complex series of gears control the arms. When the gears are turned (i.e. the model is executed) the smaller balls move relative to one another in a way that exactly mimics the real planets. That means you can configure the orrery to make it represent the Solar System of today and then move the gears to see the configuration of the planets at a future date.

Today, computers mostly perform this type of modelling and it's a key product of the marriage between science and high-performance computing. Modern science uses mathematical models to make predictions of extremely complicated systems where theory and simple experimentation are insufficient. Climate models are one of most powerful examples. They describe the nature of global entities – like atmosphere, ice masses, land surfaces and ultraviolet rays – and the complex interactions between them using mathematical formulae. Executing a model like this means taking measurements, plugging the results into the model and then playing it out to obtain a prediction of the future (see Figure 4.11).

Figure 4.10 An orrery[29]

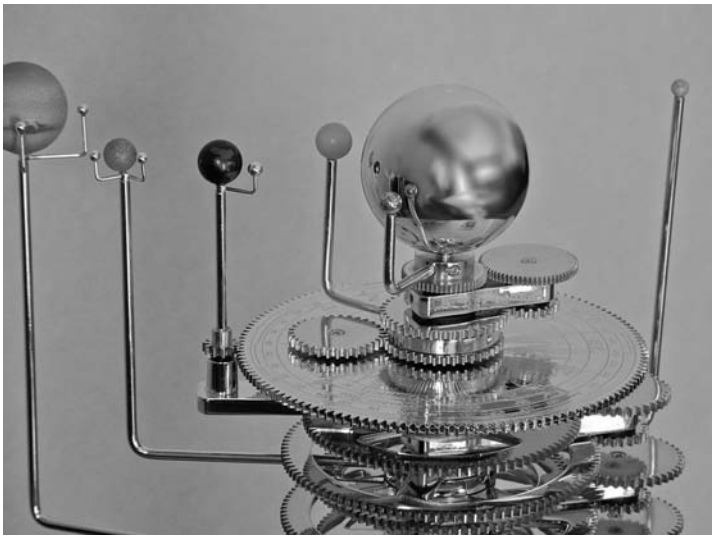
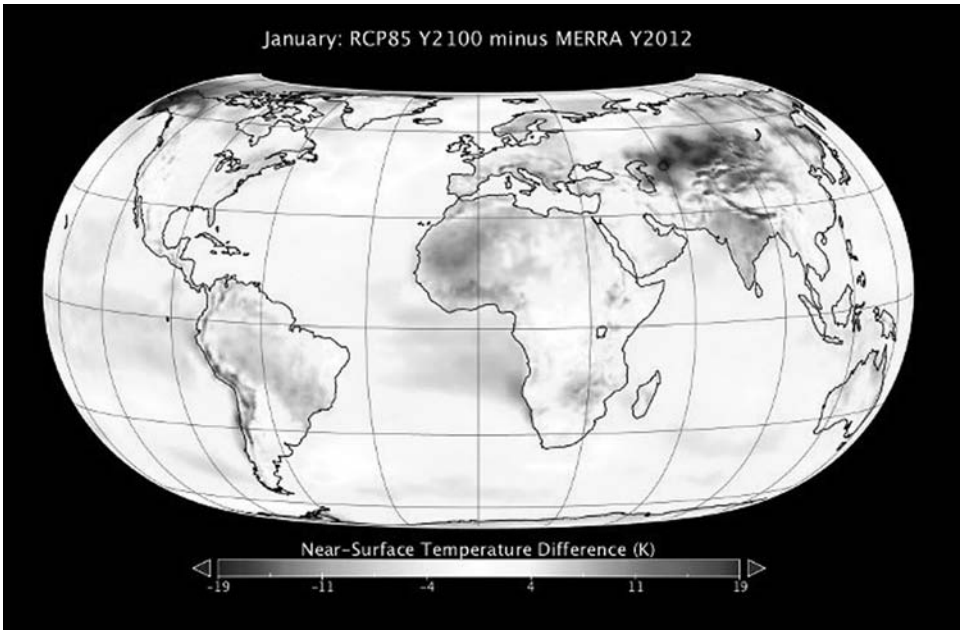


Figure 4.11 A climate model from NASA



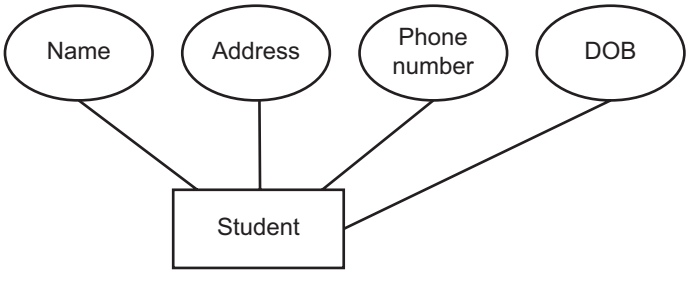
Executable models often need supplying with parameters that represent the starting state. In the case of the orrery, those parameters are the positions of the planets. In the case of the climate model, things like cloud cover, albedo, convection and so on.

Modelling data

Computerised solutions very often have to process and store data as part of their work. The algorithms that make up a solution work with that data, so they must have knowledge about the data's structure if they want to read and store it properly. As the solution's architect, it's up to you incorporate this into the algorithms.

To do this well, you should model the data before you get into the algorithmic details. This way, you'll have a much clearer understanding of the types of data your solution works with as well as the relationships between them.

Figure 4.12 Model of a student record



For example, if your task is to design a solution for storing the records of university students, there are plenty of considerations in terms of data. A simple data structure for a student record might look like Figure 4.12. With entities and relationships, this looks very much like previous examples of models. In this case, the entities are specific data fields and the relationships show how one field belongs to another. A student has a name, a date of birth, an address and so on.

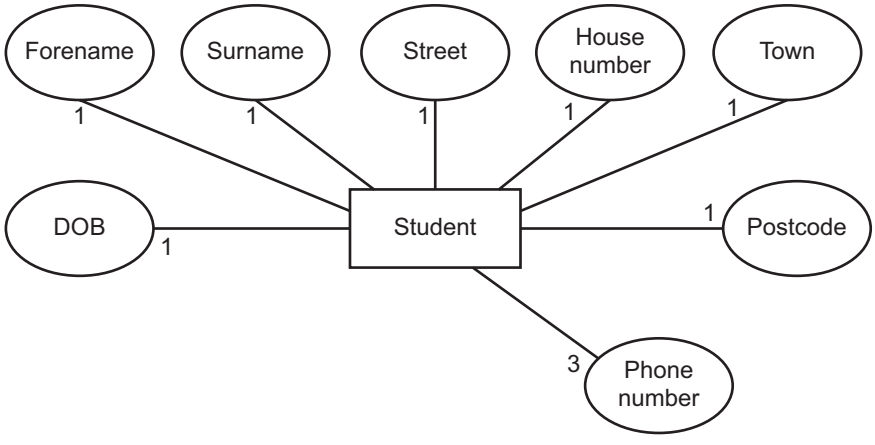
So far so good, but then you need to think about how the data will be used in your solution. Perhaps at some point you'll want to distinguish between students who live locally and those from out of town. That tells you the address field needs breaking down into constituent parts to enable the eventual algorithm to find out which city a student lives in.

Furthermore, you might wish to store multiple phone numbers for each student. To model this, you can add **cardinality** to a relationship, which tells you how many instances can be involved. In this example, a student mostly has one instance of each data item – with the exception of phone numbers.

Cardinality: a property describing the number of elements in something.

After taking these considerations into account, the revised data structure looks like Figure 4.13. The address has become a more complex entity, made up of several subfields. The cardinality added to the relationship between student and phone number states that each student record has space for up to three phone numbers.

Figure 4.13 Expanded model of a student record



Modelling interaction

If your solution involves some kind of user interaction, then you can (and should) model the interaction as early as you can. There are different ways to do this, including:

- Take the data model and pick out which data the user will see and manipulate. This helps to define the various views available to the user. For example, the data model of the student record system (see p. 261) suggests the following views:
  - personal details (name, address, telephone number);
  - course overview (a table listing each course the student is enrolled on);
  - grade overview (a table of the student's grades).
- Model the navigation. If your system provides a lot of views, the user will need a way to navigate between them all. There are numerous guidelines to follow to achieve a good user navigation. For example, the user shouldn't have to navigate through an excessive number of views to reach the one they want.
- Prototype the user interface. By creating a mock-up of the expected interface layout, you can validate it against usability standards. The process need not be at all sophisticated. In fact (where appropriate), paper prototyping is a recommended approach, wherein you draw the individual parts of the user interface onto little pieces of paper. You can then arrange them at will on one large piece of paper that represents the area of the whole view. What's more, you can show these to potential users to get feedback and collaborate with them in rearranging the pieces to improve layout.[30]

Things to remember

A few general hints and pointers to keep in mind when you carry out your own modelling are covered in this section.

Purpose

Your primary concern should be that the model is useful. The main purpose of a model is to convey an idea. If modelling something genuinely helps you to form your ideas or to communicate them to other people, then it's worthwhile effort.

Formality

Some types of models have very specific rules about how to use them. Certain symbols might have specific meaning; entities might be allowed a maximum number of relationships and so on. These are called formal models.

There's a lot to be said for more informal models, where rules are much looser, especially when you're playing with ideas or chewing things over with colleagues at a whiteboard. But there's also a time and a place for formal models. Their stricter rules are important when validity and correctness are big concerns. Furthermore, standardised formal models are essential for the smooth exchange of information between different parties.

Accuracy

> All models are wrong, but some are useful.
>
> (Box, 1987)

All models are, to varying extents, approximations. They're our attempt at taking real-world phenomena and making simplified representations. One question to ask when making a model is: what level of accuracy is acceptable? Any modern map, no matter how detailed, is not totally accurate. Consider the coastlines. The boundary between the land and the sea changes hour to hour as the tides advance and retreat. The shape of the coastline is reflected at something like the kilometre scale. No map depicts the shape of the land right down to the tiny cracks on cliffs or pebbles on the beach, but that doesn't matter for everyday navigation. A model may not be perfectly accurate, but it can be perfectly sufficient.

Precision

Values in the real world can be thought of as continuous. There are an endless number of possible values between a temperature of 24.1 °C and 24.2 °C. No matter how precisely you measure a temperature (for example, 24.134 °C), you could always be more precise (24.1342 °C). However, computers deal with discrete values, that is, values which are distinct and individual. Computers store numbers by turning on or off a small group of transistors. As a result, they have maximum/minimum values and can be precise only to a certain number of decimal places.

When modelling continuous, real-world phenomena, we are usually constraining them, putting them into a more restrictive discrete box. It's up to you to judge how precise your model needs to be. Be mindful of the context and let it guide your judgement.

SUMMARY

This chapter introduced abstraction as a key driver behind helping to understand solutions and make them more powerful. By picking out only the relevant details and suppressing the remainder, you can represent only what it is essential to a problem, leaving you free to concentrate on solving the essence of the problem. By using layers of abstraction, you can consider a problem from multiple points of view, while still benefitting from a reduction of mental load.

Modelling, a type of abstraction, plays an important part in problem-solving by representing the key parts of a solution. This helps you to understand the system you're trying to build, as well as predict its eventual behaviour. Models come in numerous forms, which you can freely choose from.

EXERCISES

EXERCISE 1

Mark the following statements as true or false:

A. An abstraction leaks when it takes details into account unnecessarily.
B. An abstraction must be instantiated before it can be executed.
C. A dynamic model shows the changes in state of something over time.
D. A directed graph is a dynamic model.
E. Accuracy describes closeness to the true value; precision describes the level of refinement in a measurement.

EXERCISE 2

Order the following into layers of abstraction, starting with the most general and ending with the most specific.

A. penguin;
B. bird;
C. Tommy the Penguin;
D. animal;
E. Emperor penguin.

EXERCISE 3

For renting out a car, which of the following is it necessary for a rental company to know about? For each one, explain your answer:

A. the car's current mileage;
B. the customer's height;
C. number of wheels on the car;
D. the car's current fuel level;
E. ID number of the customer's motorcycle licence.

EXERCISE 4

A shopping website allows you make purchases online and have them delivered to you. Here are the facts:

A. You can create an order and use it to compile a list of items.
B. When you're happy with an order, you place it with the system.
C. Eventually, the people running the site look at your order to confirm it can be fulfilled. If it can, it is confirmed and the user is informed by email.
D. When the goods are packaged and sent, the order is dispatched.
E. The order can be cancelled at any time prior to confirmation.

Model all the possible stages of an order using a state machine diagram.

EXERCISE 5

Look again at the final diagram in the section 'Modelling data'. Extend the diagram so that the data structure records the student's grades. A student is enrolled on up to eight courses maximum and each course has a predicted and actual grade.